KG

PORTFOLIO

Kristopher Gallow

AI-native operator and systems builder. Senior Capital Underwriter with a Process Automation focus at Toast, now on its Non-R&D AI Initiatives team, plus nine LLM-powered tools shipped through Ghost Strategies LLC.

Live Site LinkedIn GitHub

9+
Years in financial services

18
Production tools shipped

$500M+
Capital deployments supported

FEATURED WORK

EA

Earnings Analysis AI Agent

Ghost Strategies · Autonomous LangGraph agent

Independent

A LangGraph state machine that behaves like an analyst, not a script: a router inspects each earnings release at run time and sends clean data down a free deterministic path while messy prose goes to the LLM. Every extracted figure is validated by real code. Failures loop back with the errors injected into the retry prompt, capped at three attempts before failing loud to a human. Nothing ships without explicit approval; the graph pauses to ask. Live market data flows in from multiple providers: yfinance, Alpha Vantage, and Polygon.

Stack Python, LangGraph, LangChain, OpenRouter **Design** Code-validated · bounded retries · human-approved

PT

Ghost Strategies Portfolio Terminal

Independent

Ghost Strategies LLC · RIA portfolio analysis · three-part system, v1.6

A Bloomberg-style portfolio analysis terminal for a registered investment advisory practice, built as three parts: a single-file HTML terminal (~4,400 lines, six tabs) that ingests holdings from a workbook or raw brokerage exports and computes returns, beta, correlation-matrix standard deviation, Sharpe, drift, look-through concentration, tax-aware rebalance moves, and 2008/2000/2020/2022 stress scenarios entirely client-side; a Cloudflare Worker that fronts four market-data providers and five AI providers (eight models) behind token auth and KV caching; and a Python ingest pipeline that pulls ten years of price history and writes the results into the HTML in place. The AI Analysis tab renders a severity-sorted signal board and narrative beside a data rail drawn only from the terminal's own numbers, under an analysis-not-advice compliance frame. Verified by 111 headless assertions and a 130-agent adversarial audit that surfaced and fixed 56 defects.

Stack Vanilla JS, SheetJS, Cloudflare Workers, Python (yfinance, pandas)

Verification 111 automated checks · 56 audit findings fixed

CM

COO/COLE Case Management Dashboard

Toast

Toast · 10-person underwriting team · 700+ active cases

The team's 700+ row spreadsheet became a full application without leaving Google Workspace: a live case table with optimistic-UI edits, 15+ chart analytics including a cohort-bucketed Bizops funnel that separates queue-wait from rep turnaround, and a per-rep diagnostics panel that reconciles dashboard counts against raw sheet countifs. It now largely runs itself: nightly auto-expiry and monthly history snapshots on scheduled triggers, a Needs Attention queue for slipping cases, and no-code configuration so team leads administer everything from Settings without a developer in the loop. Leadership answers ad-hoc questions with a built-in SQL-style Data Lookup, and its case export feeds a separate tool for Legal and Risk, the COO/COLE Reconciler.

Stack Apps Script, JavaScript, Tailwind, Chart.js (2 custom plugins) **Users** 10

RC

COO/COLE Reconciler

Toast

Toast · Legal and Risk · application-to-completion reconciliation

A single HTML file that tells Legal and Risk which submitted applications actually ended in a completed COO, COLE, or both, by joining a Snowflake applications export to the dashboard's case export on the restaurant GUID. Every application lands in one of eleven written outcomes, so each one that did not complete has a stated reason, and the files never leave the user's computer.

Stack Vanilla JavaScript, SheetJS, Intl API **Users** Legal and Risk

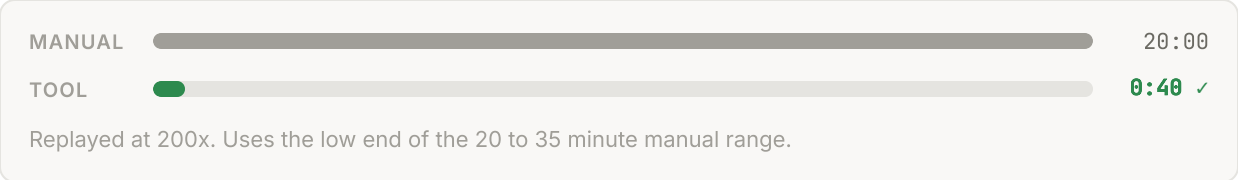
FV

Funding Verification Tool

Toast

Toast · Capital Underwriting · 20-35 minute workflow cut to 40 seconds

A single HTML file that replaced a fragile VLOOKUP chain: drag in four source files and a hash-join engine reconciles hundreds of thousands of rows in milliseconds across ten tabs, with processor-specific match logic isolated in code instead of formulas. Large bank files parse through a persistent three-worker pool with zero-copy transfers, and exports edit a bundled template's XML at runtime so the master sheet's conditional formatting survives. Successive automation passes then closed the manual gaps: a worklist that finds every row still carrying placeholder banking data, routing numbers verified against the Federal Reserve's FedACH directory and resolved to bank names in-app, and account flags applied straight from the source file instead of by hand. A 20–35 minute batch became 40 seconds.



Replayed at 200x. Uses the low end of the 20 to 35 minute manual range.

Stack Vanilla JavaScript, SheetJS, custom RFC 4180 parser **Sources** 4-way JOIN across 10 tabs

UC

Underwriting & Collections Toolkit

Toast · Capital Underwriting & Collections

Toast

Three client-side tools: a lien docket tool that de-dupes and type-classifies UCC & tax-lien filings with per-row badges, shipped as both a dashboard and an in-page bookmarklet that run the identical parse/classify engine so results never diverge (the bookmarklet runs it against the logged-in data provider); a color-coded application-routing advisor; and a collections extractor that defeats an AG-Grid's row and column virtualization by stitching cells into whole rows by stable column ID, with a paste fallback that parses multi-line, JSON-valued cells. All in-browser, nothing transmitted; part of a toolset that cut review time ~60%.

Stack React, Vanilla JS, Bookmarklets **Impact** Part of a toolset that cut review time ~60%

SQ

Self-Serve Snowflake Query Helper

Toast · Capital Underwriting

Toast

Plain English in, Snowflake SQL out. A teaching layer maps each phrase of the request to the exact clause it became, so non-analysts learn while the ad-hoc-query bottleneck on senior analysts disappears. Under the hood: one normalized contract over nine LLM providers with live model discovery (new models appear with zero code changes), a single effort control translated per model family into three different reasoning mechanisms, and a token-gated, fail-closed proxy that sends only schema metadata, never a row of data. The same engine ships three ways: web app (including a fully local Ollama mode), Cowork skill, and a claude.ai artifact that bills the viewer's own subscription.

Stack Vanilla JS, Cloudflare Workers, 9-provider LLM proxy, Ollama **Forms** Web app · Cowork skill · Claude artifact

GS

Ghost Strategies Terminal Suite

Independent

Ghost Strategies LLC · LLM-powered financial tooling

Six production tools built with Anthropic's Claude, plus the practice that runs on them: Wealth Compass (a 13-sheet, 755-formula workbook parsed entirely client-side; no data leaves the device), Training Terminal (SQL, Python, JavaScript, and Rust, 200 questions graded in the page; Rust is checked against a canonical answer and is not compiled), Model Bench Terminal (blind LLM evaluation and a decision engine that assign the other tools' model slots on recorded evidence), Economic Indicators Terminal (a macro dashboard whose research desk never charts a number the model wrote; the page computes the statistics), Garage Terminal (collector car market prices refreshed by a weekly GitHub Actions scrape), and Ghost Lead Terminal (the CRM behind a governed, human-in-the-loop agent operating model).

Stack Python, JavaScript, GitHub Actions, Pyodide, sql.js

Discipline I own the logic, the LLM writes the code, I validate every output

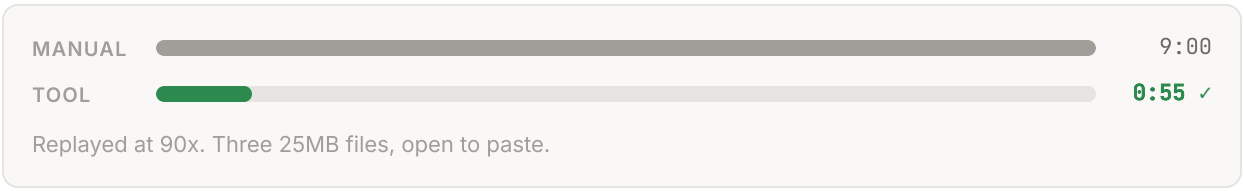
CX

CSV Column Extractor

Toast

Toast · Bizops · 9-minute workflow cut to under 55 seconds

Performance engineering wrapped in a tool simple enough for non-technical operations users: streams 1M+ row spreadsheets through parallel Web Workers, with a pipeline that starts file I/O the moment a file lands and pre-warms workers by compiling SheetJS during the user's review window. The parser constructs only the three needed cell addresses per row instead of materializing full sheets, and clipboard output ships in 150K-row chunks calibrated to stay under Google Sheets' 10-million-cell limit. A nine-minute open-to-paste workflow now takes under 55 seconds.



Stack Vanilla JavaScript, Web Workers, Papa Parse, SheetJS, Promise.all **Scale** 1M+ rows/file

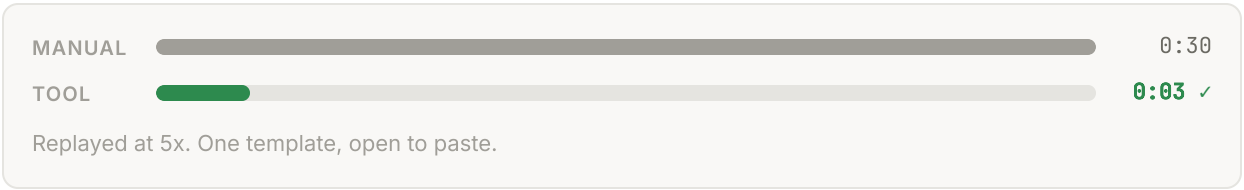
ET

Email Template Automation Suite

Toast

Toast · 14-person Capital Underwriting and Change of Ownership · 12 phases

Twelve phases from CLI script to native-feeling desktop app for a 14-person team, now with a refreshed three-pane interface: 26 built-in templates that copy with rich formatting intact (via AppleScript's «class HTML» on macOS and hand-built CF_HTML byte-offset headers on Windows), cutting retrieval from 30 seconds to under 3. Deliberately shipped as a 15KB Flask app instead of a 200MB Electron bundle, with zero AI tokens consumed and zero customer data exposure by design: placeholders fill locally, so nothing ever leaves the machine. Grew into a Salesforce-embedded generator concept presented to engineering stakeholders.



Stack Python, Flask, PowerShell, AppleScript, PyInstaller, pystray **Platforms** macOS + Windows

T

TOAST INC · SENIOR CAPITAL UNDERWRITER, PROCESS AUTOMATION

Toast Work

Nine production tools across seven projects, built for Capital Underwriting, Change of Ownership, Collections, and Bizops. Each replaced a fragile or manual workflow with a self-contained, browser-based system.

COO/COLE Case Management Dashboard

Full-stack web app · 10-person underwriting team · 700+ active cases · self-running · feeds the separate COO/COLE Reconciler

A full-stack web application built on top of a live Google Sheet (no separate backend or hosting) that replaced manual spreadsheet workflows for case tracking, analytics, and address-change requests across a 10-person team and 700+ cases. The Cases tab tracks the full lifecycle with optimistic-UI edits written straight back to the sheet, the Address Change tab manages a parallel five-status workflow on its own sheet, and the Analytics tab is an operational scorecard with team-level, per-rep, and Bizops-funnel views, all scopeable by year from a single header filter. Over successive iterations the tool evolved from a case tracker into a largely self-running system: non-technical leads administer it with zero code changes, it runs its own scheduled maintenance, and it documents itself for whoever maintains it next. Leadership's ad-hoc data questions now run through a built-in Data Lookup that brings the Snowflake Query Helper's SQL model into the dashboard, and its case export feeds a separate tool, the COO/COLE Reconciler, which gives Legal and Risk a repeatable answer to how many submitted applications actually completed a COO or COLE.

MOCK VIEW

Case Management Dashboard · Cases (47 active · 3 picked today)

47	12	4.2	98%
Active Cases	Completed (7d)	Avg Days TTC	SLA Met

CASE #	RESTAURANT	SOURCE	STATUS	DAYS
C-1042	[Redacted]	Bizops	In Review	2
C-1041	[Redacted]	Collections	Picked	1
C-1040	[Redacted]	Direct	Completed	3
C-1039	[Redacted]	Bizops	In Review	5

TRY IT: DATA LOOKUP

Data Lookup

Invented sample data. Filters work like the dashboard's, on 24 made-up cases.

Why it exists: leadership asks one-off questions, such as which restaurants completed a COO or COLE among cases submitted last quarter. The Data Lookup answers them in seconds without touching the sheet, and its SQL preview shows the logic behind any number that gets shared.

SQL preview, read-only

```
SELECT *
  FROM cases
 WHERE status IN ('Completed')
   AND submitted_on >= '2026-04-01'
   AND submitted_on <= '2026-06-30'
 ORDER BY submitted_on, case_id;
```

9	8	4	4	1
Matching cases	Unique GUIDs	COO	COLE	COO & COLE

9 rows, earliest Date Submitted first

CASE	GUID	TYPE	STATUS	SOURCE	REP	SUBMITTED	COMPLETED
C-2002	R-1002	COLE	Completed	Email	Rep B	2026-04-02	2026-04-15
C-2004	R-1003	COO	Completed	Referral	Rep C	2026-04-20	2026-05-04
C-2005	R-1003	COLE	Completed	Referral	Rep A	2026-04-21	2026-05-12
C-2006	R-1004	COO & COLE	Completed	Form	Rep B	2026-04-27	2026-05-19
C-2009	R-1007	COO	Completed	Email	Rep B	2026-05-11	blank
C-2011	R-1009	COLE	Completed	Form	Rep D	2026-05-18	2026-06-01
C-2012	R-1010	COO	Completed	Form	Rep A	2026-05-26	2026-06-09
C-2015	R-1013	COLE	Completed	Email	Rep C	2026-06-10	2026-06-24
C-2016	R-1014	COO	Completed	Form	Rep A	2026-06-22	2026-07-08

9 matching cases, 8 unique GUIDs.

Try it live: kaejhee.github.io/portfolio/#demo-lookup

KEY FEATURES

- Live case table backed by Google Sheets with an inline edit drawer, add/delete modals, and a manager Settings panel for self-serve config, plus a gated "Edit Case Details" mode that locks structural fields (Case #, Source, Type) behind an explicit toggle while status, rep, and notes stay instantly editable
- No-code administration: reps, submitters, statuses, case types, sources, address-change statuses, and the auto-expire/staleness thresholds all live in a Config tab edited through Settings, with self-healing hash-derived badge colors for newly added values, transparent schema backfills on upgrade, and off-list legacy values preserved (marked "legacy") rather than silently clobbered
- Scheduled autonomy: stale-case auto-expiry runs as a nightly time-driven trigger (moved out of the read path so reads stay write-free and cacheable), a monthly job appends summary snapshots to an auto-created History tab for durable trend and audit history, and a "Needs Attention" tab surfaces open cases past a configurable staleness threshold with a live count badge
- Performance pass: page load collapsed from four serial RPCs into one bundled call with graceful fallback, and a short-TTL server-side analytics cache keyed per year and explicitly invalidated by every write, so it stays fast without ever serving stale numbers; the cases payload is cached too (gzip-packed to fit the cache's per-key limit), and repeat visits paint instantly from the last loaded cases while fresh data loads
- Year filter: one header dropdown (YTD, All Time, or any year auto-discovered from the data) scopes both the case table and the full analytics engine, which rebases its year boundaries so past years render complete pictures; rolling 7/30-day lookbacks deliberately stay anchored to today and are labeled as such
- Analytics tab (30-day, MoM, YTD) with Time-to-Decision buckets and a Bizops/Collections funnel of three cohort-bucketed nested-bar charts (Created → Picked → Decided) that surface each month's open WIP as a visible band, separating queue-wait from rep turnaround

- Inherited-case days rebasing: for handoff cases, rep-time metrics are recomputed in-memory as (final date − pick date) so team and per-rep averages reflect rep-controlled time, not queue inflation, while system-time charts re-derive from raw dates and stay unaffected
- Rep Diagnostics panel: a per-rep deep dive (as submitter and as assigned rep) built to reconcile dashboard chart counts against sheet countifs, with row-level raw-value dumps that expose the hidden whitespace and casing that break exact-match counts
- Two custom Chart.js v4 plugins, barTotalsPlugin (per-bar and stacked totals) and nestedBarLabelsPlugin ("inner / outer" cohort labels), with disciplined chart-instance teardown to avoid canvas-reuse leaks
- Hardened Sheets-as-database layer: a bottom-up last-row scan so appends never land at row 4,000+, LockService on concurrent writes (with try/finally lock release), VLOOKUP formula copy + flush before read-back, UTC date normalization, same-day 0-day clamping, and server-side status normalization (case-insensitive canonicalization against the config list) that eliminated an entire class of data-validation rejections
- Address Change tab on its own sheet: five-status lifecycle, full add-request parity (mark Complete and "Checked in Clear" at creation), idempotent status-column and data-validation auto-creation, and unified GUID search across both workflows
- Idle-refresh banner after 30 minutes of inactivity and a version-update notifier that polls the deployed app version every 10 minutes
- Data Lookup, the Snowflake Query Helper's SQL model embedded in the dashboard: Settings was rebuilt as a leadership reporting panel that opens on it, and its filters (a date range on any of five date columns, multi-select case type and status, plus source, rep, submitter, fraud, TCA and GUID) combine like a SQL WHERE, with values OR'd within a filter and filters AND'd. A live read-only SELECT preview shows the exact logic behind any shared number; results report matching cases and unique GUIDs, copy as a GUID list or a table, and export to CSV, and an amber notice counts cases a date filter dropped for a blank date, so a result is never silently short
- Self-documenting: an in-app About/Maintenance panel carries the deploy steps, scheduled-job reminders, and conventions a successor needs, and a Help modal explains every tab and automatic behavior; the institutional knowledge lives in the app, not in one person's head

The Data Lookup's case export is one of the two inputs to the COO/COLE Reconciler, a separate tool that shows Legal and Risk which submitted applications actually ended in a completed COO or COLE.

STACK

`Google Apps Script` `HTML` `CSS` `JavaScript` `Tailwind CSS` `Chart.js v4` `Google Sheets` `google.script.run` `LockService` `CacheService` `Time-driven triggers` `Google OAuth`

COO/COLE Reconciler

Single-file HTML · Legal and Risk · application-to-completion reconciliation

A single HTML file that answers one question for Legal and Risk: of the applications submitted in a given window, which actually ended in a completed COO, COLE, or both? It joins an applications export from Snowflake with a cases export from the COO/COLE Case Management Dashboard's Data Lookup, matched on the restaurant GUID. It needs no install, server, or login, and files are processed on the user's computer and never uploaded.

TRY IT

Applications against dashboard cases

Invented sample data. The matching rules are the reconciler's own, cut down to 13 applications.

Why it exists: Legal and Risk need to know which applications submitted in a window actually ended in a completed COO or COLE, and why the rest did not. The reconciler matches each application to the dashboard's cases on the restaurant GUID and gives every application exactly one outcome.

Applications submitted in March 2026

APP	GUID	SUBMITTED
A-3001	R-2001	2026-03-02
A-3002	R-2002	2026-03-03
A-3003	R-2003	2026-03-04
A-3004	R-2004	2026-03-05
A-3005	R-2005	2026-03-06
A-3006	R-2006	2026-03-09
A-3007	R-2007	2026-03-10
A-3008	R-2008	2026-03-11
A-3009	R-2009	2026-03-12
A-3010	R-2010	2026-03-16
A-3011	R-2011	2026-03-18
A-3012	blank	2026-03-19
A-3013	R-2004	2026-03-20

Dashboard cases

CASE	GUID	TYPE	STATUS	SUBMITTED	COMPLETED	CLOSED
C-4001	R-2001	COO & COLE	Completed	2026-03-03	2026-03-20	
C-4002	R-2002	COO	Completed	2026-03-04	2026-03-10	
C-4003	R-2002	COLE	Completed	2026-03-04	2026-03-25	
C-4004	R-2003	COO	Completed	2026-03-05	2026-04-03	
C-4005	R-2004	COLE	Completed	2026-03-06	2026-03-27	
C-4006	R-2005	Other	Completed	2026-03-07	2026-03-18	
C-4007	R-2006	COO	Completed	2026-03-10	blank	
C-4008	R-2007	COLE	In Progress	2026-03-11		
C-4009	R-2008	COO	Expired	2026-03-12		2026-04-11
C-4010	R-2009	COO	Completed	2026-03-13	2026-04-19	
C-4011	R-2010	COLE	Canceled	2026-01-08		2026-02-04
C-4012	R-2012	COO	Completed	2026-03-20	2026-03-26	
C-4013	R-2013	COLE	Completed	2026-04-20	2026-05-10	

38.5%

Completion rate

5 of 13 applications ended in a completed COO or COLE inside the 30-day window.

13
Applications in the window
11 unique GUIDs

5
Completed
A COO, a COLE, or both

22
Days to complete
Median, with a mean of 19.8

2
COO & COLE
Both finished

1
COO only
Only a COO finished

2
COLE only
Only a COLE finished

3
Not completed
Every window has closed

2
No case found
Nothing to match in the window

3
Needs review
A data gap to fix first

Outcome breakdown

Each bar is one outcome's share of every application in the run.

COO & COLE completed Completed		**2** · 15.4%
COO completed Completed		**1** · 7.7%
COLE completed Completed		**2** · 15.4%
Case in progress Not completed		**1** · 7.7%
Case expired / canceled Not completed		**1** · 7.7%
Completed outside window Not completed		**1** · 7.7%

No case in window No case found		**1** · 7.7%
GUID not in dashboard No case found		**1** · 7.7%
Completed, no completion date Needs review		**1** · 7.7%
Completed, unrecognized type Needs review		**1** · 7.7%
Application missing GUID Needs review		**1** · 7.7%

2 completed applications are covered by a completed case that also covers another application, as when a restaurant reapplies inside the window. Each one counts, and its Note says so.

Each application's outcome, in submitted order

APP	GUID	SUBMITTED	OUTCOME	GROUP	DAYS TO COMPLETE	NOTE
A-3001	R-2001	2026-03-02	COO & COLE completed	Completed	18	
A-3002	R-2002	2026-03-03	COO & COLE completed	Completed	22	COO done on one case, COLE on another
A-3003	R-2003	2026-03-04	COO completed	Completed	30	
A-3004	R-2004	2026-03-05	COLE completed	Completed	22	One completed case covers this and a second application with the same GUID
A-3005	R-2005	2026-03-06	Completed, unrecognized type	Needs review		Type Other is neither COO nor COLE, though it finished in the window
A-3006	R-2006	2026-03-09	Completed, no completion date	Needs review		Marked Completed, but no completion date is recorded
A-3007	R-2007	2026-03-10	Case in progress	Not completed		Still open (In Progress)
A-3008	R-2008	2026-03-11	Case expired / canceled	Not completed		Closed without finishing (Expired)
A-3009	R-2009	2026-03-12	Completed outside window	Not completed		Finished 38 days after the application; the limit is 30 days
A-3010	R-2010	2026-03-16	No case in window	No case found		1 case found, but it was not open during the window
A-3011	R-2011	2026-03-18	GUID not in dashboard	No case found		No dashboard case has this GUID
A-3012	blank	2026-03-19	Application missing GUID	Needs review		The application has no GUID to match on
A-3013	R-2004	2026-03-20	COLE completed	Completed	7	One completed case covers this and a second application with the same GUID

Reverse check: completed cases from 2026-03-01 to 2026-04-30 (window end plus 30 days) on a GUID with no application in the window. If this list is long, check that both files use the same GUIDs and that the window is wide enough.

CASE	GUID	TYPE	COMPLETED
C-4012	R-2012	COO	2026-03-26

Ran with a 30-day window: 5 of 13 applications completed a COO or COLE.

Try it live: kaejhee.github.io/portfolio/#demo-recon

KEY FEATURES

- One written matching rule: an application counts as completed when a case on the same GUID is Completed on or after the application date and no more than 30 days after it, day 30 included (the window is adjustable), and a COO and a COLE completed on separate cases count as both
- Every application lands in exactly one of eleven outcomes across four groups (Completed, Not completed, No case found, Needs review), assigned by the first rule that fits, so each one that did not complete has a stated reason: still in progress, expired or canceled, completed outside the window, or not in the dashboard
- Built-in sanity checks: automatic column mapping with validation chips (how many application GUIDs appear in the dashboard file, and a warning when an ID column does not look like GUIDs), a reverse check listing completed cases with no matching application, and a notice when one case falls inside two applications' windows
- Time-zone aware: timestamps without an offset are read in a chosen zone, and dates are compared as calendar days because the dashboard records no times
- Shareable output: an Excel report with a summary, every application's outcome, the completed cases that have no application, and the column mapping used, plus a CSV export and one-click copy of completed GUIDs
- Tested: 138 assertions against the reconciliation core, run on fake sample files so no real data is needed to verify it

STACK

`Vanilla JavaScript` `HTML` `CSS` `SheetJS` `Intl API (time zones)` `JavaScriptCore tests`

Funding Verification Tool

Single-file HTML · 20-35 minute workflow cut to 40 seconds

A single-file HTML application that replaced a fragile VLOOKUP chain in Google Sheets used to verify merchant banking data for capital deployments. Performs an in-browser JOIN across four sources (New Funding file, Snowflake export, Original CSV funding records, and DSL bank files), produces match results with processor-specific logic, supports real-time manual overrides, and exports a multi-sheet XLSX workbook from a bundled template that preserves the master sheet's conditional formatting. The tool grew to 10 tabs, including a TCA Check tab that auto-flags Toast Checking accounts and matches pasted GUIDs against an auto-detected canonical GUID column, an External MCA Refi tab that tracks refi lineage between loans, and a Funding Edits tab that turns every row still holding placeholder banking data into an editable worklist with FedACH routing-number verification and bank lookup built in. The full funding verification workflow used to take 20 to 35 minutes per batch; the latest measured time (June 2026) clocked it at 40 seconds end-to-end.

BEFORE AND AFTER



Replayed at 200x. Uses the low end of the 20 to 35 minute manual range.

BEFORE · 20-35 MINUTES

1. Manually maintain VLOOKUP chains across four Google Sheets
2. #N/A errors propagate when Snowflake data is delayed, no path to manual fix
3. No audit trail for manual SPA verifications or Toast Checking accounts
4. WORLDPAY and CHASE rules embedded in formulas, easily broken

AFTER · 40 SECONDS

1. Drag-drop four files, JOIN runs in milliseconds across 10 tabs
2. Manual Overrides tab surfaces Check Data rows, recompute on entry
3. Toast Checking and Verified in Merchant labels persist across re-uploads
4. Processor logic in code, 4-sheet XLSX export preserves master sheet formatting

TRY THE LOGIC

Routing-number lookup

A handful of real routing numbers. The real tool searches the Fed's full FedACH directory through a local Python service.

Why it exists: funding files often arrive with a real routing number but no bank name. The Bank column reads **Unavailable**, **External**, or **UNKNOWN**, and each one used to mean a manual search to find the bank. Now the tool looks it up in place.

A funding file as it arrives

ROUTING	BANK	AFTER LOOKUP
021000021	Unavailable	JPMorgan Chase Bank, NA
026009593	External	Bank of America, N.A.
121000248	UNKNOWN	Wells Fargo Bank, NA

1 · Typo check → 2 · FedACH directory

Directory entries used in this demo

ROUTING	BANK	LOCATION
021000021	JPMorgan Chase Bank, NA	New York, NY
021000089	Citibank, N.A.	New York, NY
026009593	Bank of America, N.A.	New York, NY
121000248	Wells Fargo Bank, NA	San Francisco, CA

Found: JPMorgan Chase Bank, NA. Confirmed ACH participant, with its bank name ready to apply.

Try it live: kaejhee.github.io/portfolio/#demo-aba

MOCK VIEW

Funding Verification · Ops SPA Deals (12 rows)

EXT ID	PROC	MATCH R	MATCH A	LABEL
EXT-8821	WORLDPAY	YES	YES	Toast Checking
EXT-8820	CHASE	YES	Check Data	Pending
EXT-8819	WORLDPAY	NO	YES	Verified
EXT-8818	CHASE	YES	YES	Verified

KEY ENGINEERING DECISIONS

- Custom RFC 4180 CSV parser written from scratch to replace SheetJS's unreliable CSV mode (BOM stripping, mixed line endings, embedded quotes)
- MID normalization for JavaScript scientific notation: Snowflake XLSX exports return Merchant IDs as 1.23E+9, breaking direct string compares
- Hash-join via Map objects: equivalent to NewFunding LEFT JOIN Snowflake LEFT JOIN OriginalCsv LEFT JOIN DSL, executes in milliseconds for hundreds of thousands of rows
- Persistent DSL Worker Pool (size 3): operators routinely drop 4 to 10 large DSL files at once, so XLSX parsing runs through a fixed pool of three Web Workers fed by a queue and dispatcher, with zero-copy ArrayBuffer transfer and workers reused across drops rather than re-spawned
- Processor-specific match logic isolated in code: WORLDPAY requires exact DSL-to-file match; CHASE requires file value to differ from a known placeholder
- Real-time override injection: when a Snowflake lookup misses, manual GUID/MID/Proc Partner entry constructs a synthetic Snowflake-like object and the JOIN re-runs without a page refresh
- Labels and overrides keyed to External ID, survive re-uploads of any source file within the same session
- Template-based XLSX export: bundles the master Google Sheet's exact conditional formatting rules in a base64 template and edits the template XML at runtime via JSZip to inject the live data, preserving every YES/NO/Check Data cell fill from the original
- TCA Check subsystem: auto-detects the canonical Toast GUID column per source using a UUID-density times uniqueness score, and matches pasted GUIDs against in-batch records with per-loan and per-merchant dedup modes
- Automatic TCA labeling: rows whose source record already carries the balance-account tag are cross-referenced against the current batch and flagged Toast Checking on load; the paste path is now the exception, not the default, and out-of-batch records are deliberately left alone
- Funding Edits worklist: scans the source funding file for every row still carrying placeholder banking data (unresolved routing and account values, bank names left as a placeholder, and

external MCA refi rows), classifies each by type, opens only the cells that type is allowed to edit, and tracks per-row completion so a row clears only once its placeholders are genuinely replaced rather than merely touched. Finished rows export in one click for the tracker

- Routing-number verification against the FedACH directory: bank names that arrive as Unavailable, External, or UNKNOWN no longer need a manual search. A local Python service loads the Federal Reserve's FedACH and Fedwire directories, so each routing number is confirmed as a real ACH participant and resolved to its bank name, with a public routing API as a fallback. Routing and bank pairs already in the batch file answer first at no cost, results are cached per session, a quick checksum catches mistyped numbers before any lookup, and a configurable routing number is flagged on sight
- Refi lineage tracking: the External MCA Refi tab links each refi loan to its original (matching the External ID minus its suffix) and highlights the original loans magenta in the XLSX export
- Dual audit export: a multi-sheet XLSX workbook (Ops sheet plus additional audit sheets) and a landscape PDF audit record that reproduces the on-screen match colors and row tints for reviewers who never open the spreadsheet. Both are date-stamped at export
- 10-tab structure: Ops SPA Deals (primary output with stat-pill filter shortcuts), four source upload tabs, TCA Check, Refi, External MCA Refi, Manual Overrides, and Funding Edits
- Ships with two companion artifacts: a 13-page Standard Operating Procedure (built with a python-docx script, 11 embedded screenshots, data-flow diagram, troubleshooting guide, and a domain glossary), and a standalone Funding File Validator that round-trip-checks the exported file against the source before handoff

STACK

`Vanilla JavaScript` `SheetJS 0.18.5` `JSZip 3.10.1` `jsPDF` `Custom RFC 4180 parser` `FedACH directory lookup (Python)` `Web Worker pool` `Web FileReader API` `Base64 template embedding` `Clipboard API`

Underwriting & Collections Toolkit

Client-side underwriting & collections tools · React + vanilla-JS bookmarklets

A suite of zero-dependency, browser-based tools that replaced brittle spreadsheet macros across lien review, application triage, and collections history. Analysts paste or scrape third-party lien/UCC records (from a KYB data provider) and application details and get parsed, de-duplicated, recency-filtered, type-classified output (or a color-coded routing recommendation) to drop straight into notes. The same shared-engine pattern was extended to collections, extracting parent-account default and churn history from a virtualized data grid into a standardized, paste-ready summary. Everything runs locally, no backend, and transmits no data. Part of a set of underwriting tools that together cut review time by roughly 60%.

WHAT'S UNIQUE

- Lien Docket, one tool in two forms: the dashboard ingests pasted card text or raw API JSON, extracts per-filing fields, de-dupes, sorts newest-first, applies recency windows, and tags each filing by type with per-row badges so misclassification is obvious at a glance, with summary metrics, diagnostics, one-click copy, and .txt/.csv export; the bookmarklet runs the identical parse/classify engine in-page against the logged-in provider, sidestepping cross-origin and authentication limits
- App Router: a decision-support bookmarklet that reads live application fields and renders a color-coded review-depth recommendation
- Parent Default & Churn Extractor: reads parent-account default and churn history from a virtualized AG-Grid tracker into a standardized, paste-ready summary. It defeats row- and column-virtualization by stitching cells into whole rows by stable column ID (not screen position), with a paste fallback that record-splits multi-line, JSON-valued cells; the grid was reverse-engineered first with a read-only DOM probe
- Shared engines, not copies: the Lien Docket's dashboard and bookmarklet run one parse/classify/format engine, and the Churn Extractor's bookmarklet and paste paths share one format engine, so results never diverge; fail-loud by design: unreadable dates are kept and flagged, never silently dropped
- Config over hard-coding: keyword lists, recency windows, routing thresholds, and field labels live in editable config blocks so rule changes don't touch the logic

STACK

`React 18` `Babel Standalone` `Vanilla JavaScript` `Bookmarklets` `AG-Grid extraction` `Clipboard API` `Blob/URL export`

Self-Serve Snowflake Query Helper

Teaching-oriented text-to-SQL · web app + Worker proxy · Cowork skill · Claude artifact

A browser tool that lets non-analyst teammates draft accurate, schema-aware Snowflake queries in plain English, and learn the SQL as they go, through an explicit phrase-to-clause mapping. It removes the bottleneck of senior analysts writing every ad-hoc query while keeping execution in the user's own Snowflake session. By design it never connects to Snowflake and never sees a row of data: it handles only schema metadata (table and column names, types, comments) plus the user's generic natural-language ask. All model calls route through a Cloudflare Worker that holds provider keys server-side and normalizes nine LLM providers behind one contract. The same schema-plus-teaching engine ships in three forms: the full web app, a Cowork skill that runs on the team's existing Claude seats, and a claude.ai artifact that runs on the viewer's own subscription. The last two run with no API keys or infrastructure at all.

WHAT'S UNIQUE

- Multi-provider proxy: one normalized request/response contract over nine LLM providers (Anthropic, OpenAI, Gemini, xAI, DeepSeek, Moonshot, Zhipu, Mistral, Qwen) via an adapter pattern (seven OpenAI-compatible plus dedicated Anthropic and Gemini adapters), resolving browser CORS and removing API keys from the client
- Personal or organizational: a first-run selector configures either an organizational mode (shared team Worker, provider keys as server-side secrets, gated by a Bearer token) or a personal mode: bring-your-own-key (Anthropic or Gemini directly from the browser), your own proxy for all nine providers, or fully local via Ollama with installed-model detection and nothing leaving the machine
- Zero-key distribution: the engine also ships as a Cowork skill (runs on existing Claude seats, with adversarially hardened guardrails: execution and file-write bans plus a non-waivable PII pre-flight) and as a claude.ai artifact powered by the artifact runtime's completion API, billing the viewer's own subscription with zero infrastructure
- Self-updating model discovery: queries each provider's live model endpoint and merges it with a curated overlay (24-hour TTL), so newly released models appear automatically with no code edits, an approach built on the insight that model names churn but API families stay stable
- One Low/Medium/High effort control mapped per model family to three distinct mechanisms (reasoning-effort enum, thinking-token budget, or omitted), and auto-disabled when the selected model can't reason
- Teaching layer: every result returns the SQL plus a plain-English explanation and a three-column phrase → clause → why mapping table, reframing a query generator into a learning tool
- Compliance-driven design: no database connection, metadata-only prompts, provider keys held as server-side secrets behind a token-gated, fail-closed proxy, and a client-side PII/MNPI pre-flight that warns on emails, phone numbers, GUIDs, and account-ID-shaped strings

STACK

`Vanilla JavaScript` `Cloudflare Workers` `Tailwind (CDN)` `Prism.js` `localStorage`

9-provider LLM proxy

Ollama

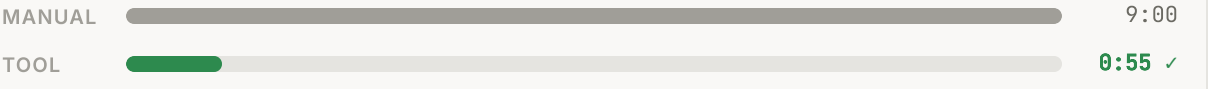
React (artifact)

CSV Column Extractor

Single-file HTML · 9-minute workflow cut to under 55 seconds

A single-file HTML dashboard for combining up to five large spreadsheet files, each potentially exceeding one million rows, and extracting three specific columns for paste into a destination Google Sheet. Iterative performance engineering took the full open-to-paste workflow on three 25MB XLSX files from approximately nine minutes (manual baseline) to under 55 seconds.

BEFORE AND AFTER



Replayed at 90x. Three 25MB files, open to paste.

BEFORE · ~9 MINUTES

1. Open three files sequentially in Excel or Sheets
2. Manually select and copy three columns from each file
3. Paste each chunk into the destination sheet, watch for cell-limit failures
4. Retry failed pastes after Google Sheets silently truncates

AFTER · UNDER 55 SECONDS

1. Drag-drop all three files into the tool
2. Pre-read and pre-warm pipeline starts in the background
3. Auto-parse fires 400ms after the last file, all three parse in parallel
4. Copy each column with one click, live cell-limit warnings prevent silent failures

MOCK VIEW

CSV Column Extractor · 3 files queued · auto-parse in 0.4s

847,302	3	~22s	3.9M
Total Rows	Files Parsed	Parse Time	Cells / Chunk

COLUMN	ROWS	CELL ESTIMATE	ACTION
Column B (MID)	847,302	22.0M cells	Chunk Required
Column L (R&T)	847,302	22.0M cells	Chunk Required
Column M (DDA)	847,302	22.0M cells	Chunk Required

Chunk size 150K rows · 6 chunks per column · safely under 10M cell limit

PERFORMANCE PIPELINE

- Parallel parsing via Promise.all: all files parse simultaneously in independent workers with local accumulators, merged in original file order after all Promises resolve
- Pre-read FileReader cache: readAsArrayBuffer fires the moment a file enters the queue, not when Parse is clicked, eliminating I/O wait from the critical path
- Pre-warmed worker pool: a Web Worker is spawned per XLSX file on drop, compiling SheetJS (about 500KB) inside the worker during the user's file-review window
- Auto-parse with 400ms debounce: no Parse button click required, handles single-file and multi-file drops uniformly, guards against re-triggering once results are displayed
- Selective XLSX cell extraction: the worker constructs only three required cell addresses per row (B+n, L+n, M+n) instead of allocating the full 2D array via sheet_to_json
- Batch and chunk sizes increased 4x: worker message batches went from 25K to 100K rows; CSV chunk size went from 5MB to 20MB, reducing postMessage round-trips proportionally

BASE ENGINEERING DECISIONS

- Papa Parse with worker: true for CSV streaming; hand-written Blob Worker for SheetJS XLSX parsing to keep the UI responsive during 1M+ row binary parses
- ArrayBuffer transferred via postMessage transferables to eliminate the memory cost of a second copy on large files
- 150K-row clipboard chunks: 150K rows times 26 default Google Sheets columns equals 3.9M cells, safely under the 10M limit
- Push-in-loop array construction instead of spread or push.apply, avoiding call-stack overflow on arrays exceeding 125K elements
- Three primary copy buttons (one per column) because the destination sheet uses non-adjacent columns; combined tab-separated copy preserved as a secondary option

STACK

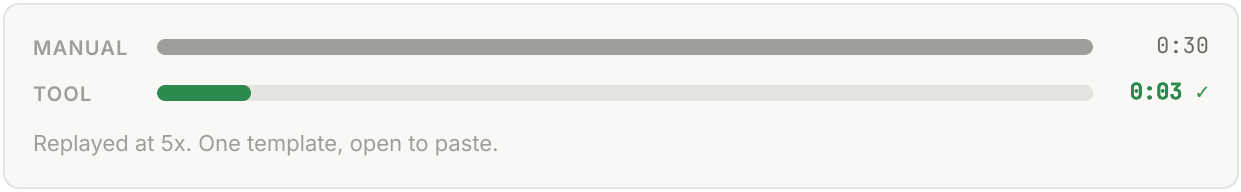
`Vanilla JavaScript` `Web Workers` `Papa Parse 5.4.1` `SheetJS 0.18.5` `Blob Workers` `Promise.all parallelism` `FileReader API` `Clipboard API`

Email Template Automation Suite

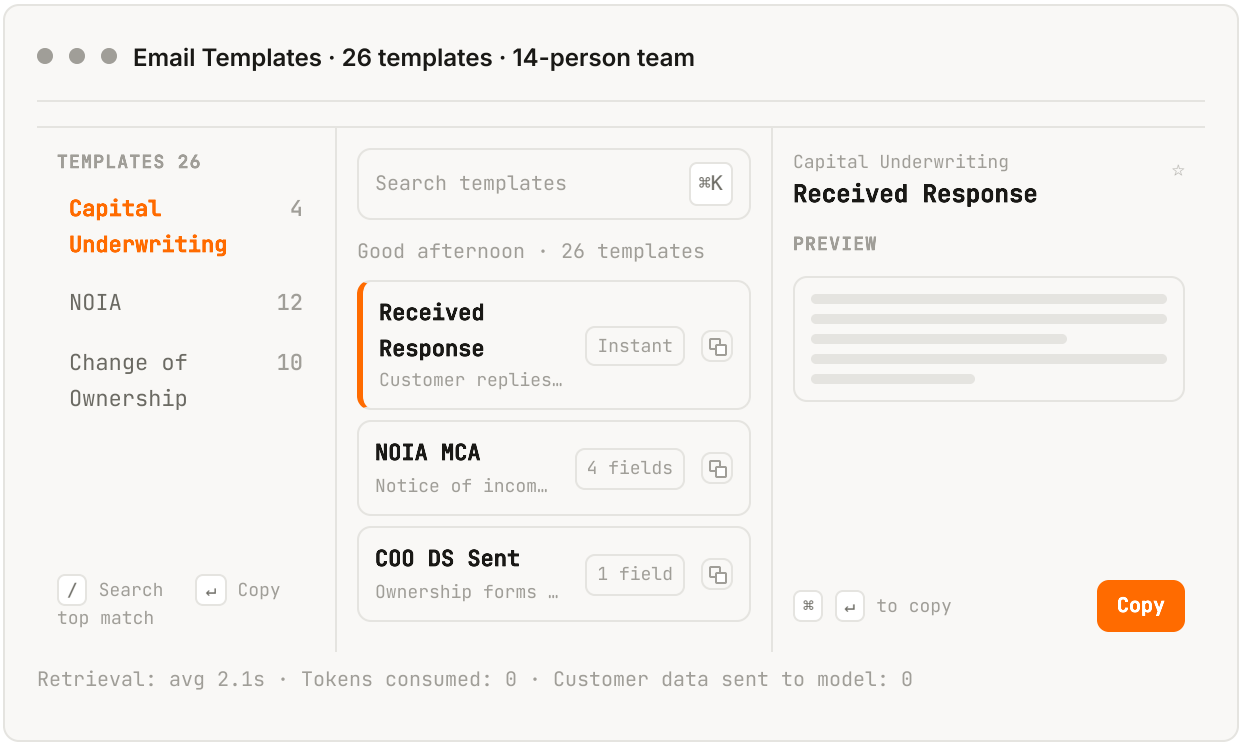
12 phases · cross-platform desktop app · 14-person team · 30s to under 3s

A cross-platform desktop application that ships 26 built-in email templates across three categories, with rich-text formatting preserved on paste. Grew over 12 phases from a CLI tool to a full Flask web app, then to a PyInstaller-bundled native application with a system tray icon and a Salesforce-embedded NOIA generator concept presented to engineering stakeholders, and most recently into a redesigned three-pane interface built around finding, filling and copying a template in a few keystrokes.

BEFORE AND AFTER



MOCK VIEW



PHASE PROGRESSION

- Phases 1 to 3: CLI tool, cross-platform GUI launcher, then a full Flask web app
- Phase 4: Dark mode, brand logo, and accordion drawer system for the editor
- Phase 5: PyInstaller-bundled standalone executable with platform-appropriate data directories (macOS app support, Windows AppData), port discovery to dodge AirPlay on macOS, and a system tray icon for clean exits

- Phase 6: Salesforce-embedded NOIA template generator concept, with function-based template rendering (eight templates, conditional logic for variable-length entity lists)
- Phases 7 to 9: Placeholder fill modal with confirmation toasts, multi-line field support, two-panel preview modal, and product demo enhancements
- Phase 10: Template-sync bug fix (silent failure mode on every launch caught and resolved)
- Phase 11: Stakeholder presentation HTML with Apps Script architecture diagrams and a NOIA automation proposal for engineering review
- Phase 12: A refreshed three-pane interface: a Toast-branded shell with a collapsible category sidebar and counts, a template list whose rows show how many fields each template needs (or Instant when none) with one-click copy, and a detail panel with a live preview, favorites and a time-of-day greeting, in light and dark themes
- Phase 12 workflow: ranked search opened with / or ⌘K where Enter copies the top match, favorites and recents for repeat sends, guided field filling with a draft kept per template where Enter moves to the next field and copies on the last, rich-text copy with a plain-text fallback, Email submitted with Undo, and custom templates written in a rich-text editor with placeholder insertion

ENGINEERING DECISIONS

- Zero AI tokens consumed by design: templates use placeholders that the user fills locally, so no customer data ever leaves the machine
- Cross-platform clipboard formatting: macOS uses AppleScript with the «class HTML» format; Windows uses CF_HTML byte-offset headers with precise UTF-8 byte counting
- 15KB Flask app deliverable instead of a 150 to 200MB Electron bundle, deliberately avoiding the Apple Developer Program signing requirement
- Per-machine identity: no authentication needed, each install is local-only

STACK

`Python` `Flask` `Bash` `PowerShell` `JavaScript` `AppleScript` `Quill.js` `PyInstaller` `pystray` `Pillow`

GS

GHOST STRATEGIES LLC · INDEPENDENT

Ghost Strategies

Nine LLM-powered tools built independently, and a practice that runs on them under a governed multi-agent operating model. Every project ships with the same discipline: I own the logic and architecture, the LLM writes the code, I validate every output, and I only prompt with test or sample data so no proprietary information ever enters the model.

HOW THE PRACTICE RUNS

Ghost Strategies runs its own operations (research, drafting, content preparation, and tooling) on a fleet of AI agents across multiple runtimes, including Claude Code, Cowork, and a self-hosted research agent. It is designed around one constraint: my review minutes are the bottleneck, so every lane minimizes them rather than maximizing agent output. Total planned operating overhead is about 80 minutes a week.

- A human stands between generation and the public: agents research, draft, and package, and nothing sends, posts, or publishes without an explicit human action
- Zero client PII to any model, in any lane, for any purpose; agents work from public data and tokenized templates only
- Verification over volume: a small batch of sourced, cited leads beats a large unverifiable one, and every factual claim in content meets the same bar
- Agents write batch files and drafts only; the CRM's validated import gate and human review are the only ways into the system of record
- Budget discipline: agent lanes run on capped, monitored spend, and overruns are fixed with output limits and model routing, not more money

Earnings Analysis AI Agent

Autonomous LangGraph agent · validate in code, retry with feedback, human-approve

An autonomous "earnings analyst" built as a self-directed project (2026): give it a ticker and it fetches live earnings data, decides on its own how to parse it, extracts revenue and EPS, validates the numbers with deterministic code, retries on failure with the errors fed into the next prompt, and pauses for human approval before shipping a beat/miss summary. It demonstrates the part of AI engineering that matters in production: not calling a model, but making its output trustworthy. Being rolled into the portfolio-analysis platform as its earnings-analysis module, scoped to public earnings data only.

```
START → fetch → route_by_shape → (format_fast | extract) → validate ↻ → human checkpoint → synthesize → END
```

STEP THROUGH A RUN

Replay the graph

Scripted replay with sample data for a made-up ticker. Node names, log lines, and outcomes match the agent's code.

fetch → route_by_shape → format_fast | extract → validate → ⏸ checkpoint → synthesize | fail_loud

```
[fetch] DEMO release loaded (sample data). It arrives as prose: "Quarterly revenue rose to $4.2 billion; diluted EPS was $1.31."
[route_by_shape] input does not start with "{" → extract (LLM path)
[extract] attempt 1 → {}   (reply was not valid JSON, so the node returns empty)
[validate] errors = ['revenue missing', 'eps missing']
[route] 1 of 3 attempts used. Back to extract with the errors added to the prompt.
[extract] attempt 2 → {'revenue': 4200, 'eps': 1.31}
[validate] errors = []
PAUSED. Extracted numbers awaiting approval: {'revenue': 4200, 'eps': 1.31}
Approve? (y/n)
[synthesize] Revenue $4.20B, EPS $1.31. Extraction validation in 2 attempt(s).
```

Try it live: kaejhee.github.io/portfolio/#demo-agent

WHAT'S UNIQUE

- Self-correcting, not one-shot: deterministic validation checks every extracted figure, and failures loop back to the extractor with the errors injected into the retry prompt, capped at 3 attempts; after that it fails loud to a human instead of shipping a wrong number

- The model generates; code judges. The AI never grades its own homework
- Human-in-the-loop by construction: the graph pauses (interrupt_before) and shows the numbers for approval; rejecting ships nothing
- Chooses its own path: a router inspects the data shape at run time (clean structured data takes a free deterministic path, messy prose goes to the LLM). That is the difference between a fixed script and an agent
- Production-shaped: refactored from a Jupyter prototype into a clean multi-file Python project (state, nodes, graph, CLI) with live market data from multiple providers (yfinance, Alpha Vantage, Polygon); the portfolio-terminal integration runs the reasoning model through OpenRouter, keeping it multi-model swappable without touching the graph
- Self-directed and documented: authored the build plan, a full written case study, and an interactive architecture walkthrough of the graph and every design decision

STACK

`Python` `LangGraph` `LangChain` `Anthropic Claude` `OpenRouter` `yfinance` `Alpha Vantage` `Polygon` `Human-in-the-loop` `CLI`

Ghost Strategies Portfolio Terminal

Bloomberg-style RIA portfolio analysis · client-side engine + Cloudflare Worker + Python ingest · v1.6

An internal terminal for reviewing client portfolios against target allocations and the current market environment, built for a registered investment advisory practice with privacy as a hard constraint: holdings are parsed and computed in the browser, nothing but the client name and proxy settings touches browser storage, and the only network traffic is an explicit, button-triggered call to the firm's own proxy. It is a sibling of Wealth Compass (the same client-side architecture, pointed at portfolio analytics instead of planning) and grew from v1.0 to v1.6 across a multi-session build driven by written handoff specifications with phased build orders, acceptance tests, and mock fixtures for keyless development.

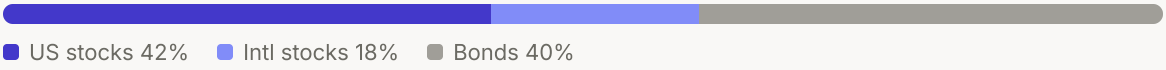
```
workbook · brokerage CSV/XLSX · manual · session.json → review grid → compute
(returns · beta · σ · Sharpe · drift · look-through · rebalance · stress) → AI
Analysis → Worker proxy → 8 models
```

TRY THE MATH

Why the terminal uses a correlation matrix Illustrative inputs for the math, not market data or advice.

Stocks vs bonds **60% stocks**

US share of stocks **70% US**



US stocks 42% · Intl stocks 18% · Bonds 40%

Simple weighted average of each asset's volatility	**12.2%**
Portfolio volatility with correlations	**10.2%**

Correlations below 1.0 cancel some of the risk. A simple average would overstate this mix's volatility by 2.0 points.

$\sigma_p = \sqrt{\sum_i \sum_j w_i w_j \sigma_i \sigma_j \rho_{ij}}$ · σ: US 16%, Intl 17%, Bonds 6% · ρ: US/Intl 0.80, US/Bonds 0.20, Intl/Bonds 0.25

Try it live: kaejhee.github.io/portfolio/#demo-vol

KEY FEATURES

- Four import paths: a structured Excel workbook, a built-in sample, manual ticker entry, and multi-file brokerage exports (CSV/TSV/XLSX) with header-alias column detection across major custodian layouts, preamble and footer skipping, per-$100 bond quoting, cash sweep lines kept as CASH holdings, and an editable review grid where nothing is committed until the advisor

confirms; a session file captures portfolio, targets, workbench proposal, and AI runs for drop-to-resume

- Computation engine, all client-side: weighted returns across five horizons, portfolio beta, standard deviation via the full correlation-matrix formula, Sharpe, weighted expense ratio, and target-versus-actual drift by asset class, category, sector, geography, and account, with per-account drill-down that recomputes every metric
- Look-through concentration: direct position weight plus the same name held inside index funds, flagging names that breach the threshold only through fund overlap
- Tax-aware rebalance engine that pairs overweight and underweight categories into dollar moves, trims tax-advantaged accounts first and then the smallest unrealized gain, and never takes more from a position than it holds; scenario stress tests (2008, 2000, 2020, 2022) applied by asset class, both index-based and beta-adjusted
- AI Analysis tab: the computed portfolio state plus live quotes and news go to a user-selected model. The choices are Claude Opus 5, Sonnet 5, or Fable 5.1 (with web search), GPT-6 Astra, Grok 4.6, Kimi K3, DeepSeek V4.1 Flash or V4 Pro. The model returns five fixed sections plus a structured JSON appendix, rendered as a severity-sorted signal board with ticker and category chips that jump to the relevant tab; every chart comes from the terminal's own calculations, never the model
- Adjustment workbench: per-category target sliders with live drift bars and implied dollar moves through the same rebalance engine, applied as clearly labeled session targets so every tab and the next AI run reason about the proposed shape; run history keeps ten runs with side-by-side comparison, plus Markdown, standalone HTML, and print export
- Compliance by construction: a verbatim analysis-not-advice system prompt, a non-dismissible internal-use banner, a footer on every result and export, news passed to the model inside explicit untrusted-text delimiters, and no automatic model calls anywhere
- Cloudflare Worker proxy: four market-data providers and five AI providers behind shared-token auth and encrypted secrets, KV caching that never caches empty results, ticker validation and encoding, provider fallback chains for quotes, refusal and truncation surfaced from every model provider, and a health endpoint the terminal uses to gate model choices
- Verification: two headless Node harnesses (111 assertions) covering parsing, computation, tax-aware rebalancing, look-through, stress scenarios, and structured-output parsing; then a 130-agent adversarial audit (nine parallel reviewers, each finding challenged by two independent verifiers) confirming 56 defects, all fixed and reproduced by a second pass, including escaping every render sink that carries imported or model text and validating session files on restore

STACK

`JavaScript` `HTML` `SheetJS` `Cloudflare Workers` `Workers KV` `Python` `yfinance` `pandas / numpy` `Anthropic Claude` `Multi-model routing` `Node test harness`

Economic Indicators Terminal

Bloomberg-style macro dashboard · grounded LLM research desk · single HTML file · v2.2

View source on GitHub: KaeJhee/economic-terminal ↗

A dark, data-dense economic terminal with seven views (Overview, CPI detail, PPI detail, employment, projections, a correlation matrix, and a News & Analysis research desk) in one HTML file of about 3,900 lines. It boots from an embedded snapshot of official releases and can replace the headline series with live FRED when a key is saved. The correlation matrix is computed in the page from the quarterly levels it is showing. Chart.js 4.4.1 ships in the repo, so the charts render without the CDN. The research desk follows one rule: *the model never supplies a number that gets charted. Statistics are computed in the page.*

WHAT'S UNIQUE

- Headline cards are the latest monthly observation, not a quarterly average. FRED dates are parsed from the YYYY-MM-DD string, so the calendar month stays put west of UTC. Real GDP is on the page from BEA's series as carried on FRED; the badge says BEA only after the BEA API returns NIPA T10101 line 1. The six summary cards are August 2026. JOLTS in the snapshot is July 2026, and real GDP runs through 2026-Q2. CPI Education, Recreation, and Other Goods, most PPI component rows, and the employment sector table are still March 2026 and are labeled on the page
- Grounded LLM pipeline: a router classifies each question into a structured plan, the page fetches prices and computes every statistic (returns, CAGR, volatility, drawdowns, divergence windows), and a web-grounded narrator explains those date windows, cited and streamed token-by-token, with the chart shading the windows the math found. Current-events claims need a source. With no OpenRouter key, the tab explains that design and shows a labeled example layout. No recorded News run is embedded
- Tiered routing by query difficulty: fast, standard, or deep narrator chosen per question. After a real run, the answer names its model and shows that query's cost
- Price history tries Twelve Data first, then Alpha Vantage compact (about the last 100 sessions), then Stooq on localhost only. Each provider failure is named with the reason. Mismatched price-adjustment bases skip pair statistics rather than chart a distorted comparison. Projection bands are a fixed illustrative formula and are labeled that way on the chart
- The News tab caches the price and FRED series it fetched in IndexedDB for 20 hours. The dashboard refresh does not. A News answer stays in memory and does not survive a reload. Google Fonts still come from Google; the charts do not
- Keys stay in localStorage. Connection-test errors are escaped, and News markdown is escaped before links render (http and https only). Both system prompts tell the model to ignore instructions embedded in the question or in web results. GitHub Actions runs a secret scan and the 37 embedded tests, with the suite in America/Chicago. The pre-push hook stays off until you run sh scripts/install-hooks.sh. The Cloudflare Worker in the repo is an example template for the live site; it is not deployed from the project. On localhost, server.py proxies FRED

STACK

Vanilla JavaScript | Chart.js 4.4.1 (vendored) | IndexedDB | OpenRouter | FRED / BEA APIs | Twelve Data | Alpha Vantage compact | Stooq (localhost) | Python localhost proxy | Example Worker template

Model Bench Terminal

Blind LLM evaluation with code judges, planned batches and a decision engine · model defaults decided per task, on the record

A single-file evaluation bench that turns model selection from vibes into recorded evidence. One prompt broadcasts to 2–4 models chosen from a live catalog of ~340; responses stream side by side with live time-to-first-token, throughput, and cost; votes are cast *blind* (panes shuffled, and names, costs and speed hidden until the vote locks); and answers with an expected result are graded by code instead: strict JSON-schema checks, field-by-field answer matching, and unit tests run in a locked-down sandbox. An Evaluate mode runs a frozen test set against a baseline and up to three challengers as a planned batch: its size and rules are recorded before the first request, and the planner states the chance the batch can reach a decision before anything is spent. Subjective answers wait in a blind rate-later queue, one keypress per judgment, and names appear only once the whole batch is rated. Default slots are decided one task at a time, from head-to-head shares with a 95% plausible range and at least 12 prompts behind them, never from a pooled win rate. A decision engine turns that evidence into a verdict under a written policy: it recommends a new champion only from a completed planned batch, when a challenger is proven better or proven no worse and at least 20% cheaper, and it blocks one shown to fail more often than the champion. Promotion still takes a click and a one-line rationale; every promotion, hold and override (flagged as an override) goes into an append-only decision log, and a decision is flagged for re-evaluation when the ground shifts. The models that fill the other Ghost tools earn their slots this way, and each task's champion exports as a model card and a defaults file for those tools.

TRY A ROUND: BLIND VOTES AND CODE JUDGES

Blind votes and code judges

Recorded sample responses from three illustrative model tiers. Timings, costs and the starting evidence are illustrative. The grading rules are the real tool's, cut down to these three prompts.

`structured-json` Extract revenue (USD millions) and diluted EPS from: "Quarterly revenue rose to $4.2 billion; diluted EPS was $1.31." Reply with JSON only: {"revenue": number, "eps": number}

A Frontier tier — FAIL format	B Fast tier — FAIL schema	C Mid tier — SCHEMA PASS
`Here is the extraction: {"revenue": 4200, "eps": 1.31}`	`{"revenue": "$4.2B", "eps": 1.31}`	`{"revenue": 4200, "eps": 1.31}`
text outside the JSON (strict parsing)	revenue: expected number, got string	
TTFT 1.1s · total 2.9s · $0.0062	TTFT 0.3s · total 0.9s · $0.0001	TTFT 0.5s · total 1.6s · $0.0009

Graded by code, so no vote is needed: C passed. Names and costs show once grading is done, and each result joins the pass rates below.

HEAD-TO-HEAD · FINANCE-NARRATIVE

share of matchups won, ties count half · a lean shows from 12 prompts

MODEL	SHARE	PLAUSIBLE RANGE (95%)	PROMPTS
Frontier tier	65%	33% to 87%	11 of 12
Mid tier	56%	27% to 82%	11 of 12
Fast tier	23%	7% to 56%	11 of 12

GRADED BY CODE · STRUCTURED-JSON AND CODE — pass rate, each prompt counted once

MODEL	PASS RATE	PLAUSIBLE RANGE (95%)	PROMPTS
Mid tier	82%	48% to 96%	11 of 12
Frontier tier	64%	32% to 87%	11 of 12
Fast tier	36%	13% to 68%	11 of 12

Try it live: kaejhee.github.io/portfolio/#demo-bench

WHAT'S UNIQUE

- Blind integrity treated as a security property, not a UI feature: every channel that leaks model identity (control-rail slugs, provider error text, history markers) is masked until reveal (blanked, not blurred, which survives select-copy)
- A cost chain that refuses to guess silently: exact stream usage → generation-endpoint lookup (captures web-search fees and pre-abort billing) → catalog estimate visibly tilde-marked; unknown costs render as a dash, never a confident $0.0000
- Ledger schema designed to be frozen: schema version and UUID from day one, per-pane finish reason and serving provider captured because they can't be backfilled; the governance data lives in a separate database, so the original file still reads the same ledger
- Code judges before opinions: extraction, JSON and code prompts carry expected answers, schemas or unit tests, graded in the page with no dependencies; unit tests run in a sandboxed worker with no network or storage access and a hard time limit, so a runaway answer times out instead of freezing the page
- Governance with a paper trail: votes and grades are append-only, test cases are versioned and only enter a frozen test set once they pass their own self-test, a disputed grade quarantines its case, and a changed grader can regrade every stored answer for free with no model calls
- Planned evaluations that resist cherry-picking: a batch is recorded with its size, test set and policy before its first request, a sample that ran is never drawn again, and names stay hidden until every

pair is rated (an early reveal is allowed, but later votes are then marked revealed and no longer count as blind evidence)

- Honest planning: before each batch the planner states the real chance it can prove an edge under the written policy (about 60% for 24 prompts rated 3 times against a 70/30 edge), instead of promising a decision a small batch cannot deliver
- Decisions under a written policy: each challenger is compared with the champion alone, reliability is checked before quality, and when none is proven better the cheapest one proven no worse and at least 20% cheaper wins; every verdict has one plain sentence and an evidence card with each range charted against the bar it must clear, the reliability check, the policy in words and how much more evidence would settle it
- Drift watch: every run records the model and provider that actually served it, and each catalog refresh is compared with the last, so a price move of 10% or more, a removed model, an unfamiliar served model, a new test set version or evidence older than 60 days opens a re-evaluation alert; a re-baseline reruns the frozen set on the champion alone and either confirms it or marks it degraded
- An evidence health panel audits the votes themselves (leftmost-pane bias, checked only once there are 30 blind picks; tie and all-bad rates; how much of the evidence came from fully blind votes), beside an exploration chart of each model's chance of beating an average model, kept separate from decisions
- Rubric checklists for subjective tasks: 3 to 5 pass/fail criteria per task, some pre-checked by code (word limits, required links); a human override is recorded, and a pre-check that disagrees with the rater too often is flagged as unreliable
- Ranges that hold up: the plausible range on each share was checked by simulation to cover the true value at least 94% of the time across 2-, 3- and 4-way comparisons, ties, reruns and one-sided results
- Hand-rolled SSE streaming with a per-pane AbortController and Promise.allSettled isolation, so one pane's mid-stream network death can't block or corrupt its siblings; escape-first rendering keeps hostile completions inert
- Audited and tested before first real use: independent code audit (15 findings resolved, including three schema gaps that would have been unrecoverable after data accumulated), headless end-to-end simulation against a stubbed API, and adversarial unit tests for the validator and renderer; each governance phase then shipped only after repeated adversarial review rounds and rollback checks against every earlier build sharing the same data (the decisions phase: four review rounds and a 146-check upgrade and rollback rehearsal across all five builds)

STACK

`Vanilla JavaScript` `OpenRouter API` `IndexedDB` `Chart.js` `SSE streaming` `Zero build step`

Training Terminal

Browser coding trainer · SQL, Python, JavaScript, Rust

View source on GitHub: KaeJhee/training-terminal ↗

A browser coding terminal (xterm.js) for SQL, Python, JavaScript, and Rust. 200 questions: 50 per track across five gated tiers (Introductory, Amateur, Intermediate, Experienced, Master), 10 questions per gate, are graded in the page. SQL runs through sql.js against a seeded database that is rebuilt between the answer and the expected query. Python runs through Pyodide in a worker, loaded only when that track starts. JavaScript answers run in a fresh worker. Network and storage APIs are deleted on the worker global and its prototypes before the answer runs, and the page's Content-Security-Policy limits connections to this site and jsDelivr. It is a practice sandbox, not a sealed one, and not a place to paste secrets. Rust is checked against a canonical answer after whitespace, comments, and a few equivalent spellings are normalized. It is not compiled. Progress, attempts, accuracy, and streak stay in localStorage, with a this-session clock and an engine-status header. Educational coding tool, not investment advice.

WHAT'S UNIQUE

- Four grading paths, all in the page: sql.js on a seeded database rebuilt between the answer and the expected query (SQL), Pyodide in a worker that loads only for that track (Python), a fresh Web Worker per JavaScript attempt, and a canonical-string compare for Rust, with no compiler in the page
- Real xterm.js REPL: a command bar (start, hint, concepts, cheatsheet, examples, schema, reset, export/import), multi-line editing, command history, and emacs-style key bindings
- Learning layer baked in: per-tier concepts auto-surface on first entry, a toggleable cheatsheet panel, worked examples, and per-question hints
- Written with AI coding agents and checked by programs: CI grades the SQL, Python, and JavaScript answer keys in the real engines, and Rust's 319 accept/reject cases run through the same grader the page uses. Ten-question gates unlock the next rank. Progress stays in this browser, with no backend

STACK

`Vanilla JavaScript` `xterm.js` `Pyodide` `sql.js` `Web Workers` `localStorage`

Ghost Lead Terminal

Single-file CRM · pipeline, outreach, and a validated import gate

A single-file HTML CRM that runs the practice's client pipeline end to end: a kanban board by stage, a sortable table with a full record drawer, a Today view for due and stale follow-ups, and a dashboard with honest funnel math. Data lives in a synced JSON file (no backend, no vendor lock), with conflict-merge prompts when two devices disagree. Agent-produced lead batches can only enter through a validated import gate, and outreach ships through a template system with a Gmail handoff and an explicit send-confirmation step, so nothing leaves without a human action.

WHAT'S UNIQUE

- Validated import gate with dedupe: agent research lands as schema-checked batch files, reviewed and merged deliberately; agents never write to the live data file
- Sync-aware single file: JSON data file synced across devices with a conflict-merge prompt instead of silent last-write-wins
- Two-click outreach flow: template fill → Gmail handoff → explicit "Email Submitted" confirmation, keeping the human send gate permanent
- Source attribution built into the schema so inbound-channel funnel math stays honest
- 24-check smoke-test suite beside the tool; schema field names treated as an additive-only contract

STACK

`Vanilla JavaScript` `Single-file HTML` `Synced JSON store` `Smoke-test suite`

Wealth Compass Terminal

Client-side financial planning dashboard

A client-facing financial planning dashboard that parses a 13-sheet workbook with 755 formulas entirely client-side. All data stays on the user's device. No backend, no data transmission, no model exposure to client information.

STACK

`JavaScript` `HTML` `SheetJS` `Custom formula engine`

AI-Assisted Financial Modeling Suite

Dynamic financial engines built with Claude

`Valuation model source: KaeJhee/valuation-model ↗`

A separate workstream under Ghost Strategies focused on financial modeling rather than terminal apps. Includes a probability-weighted valuation model and a $3M TRS retirement planner. I own the financial logic and architecture while directing Claude to generate the code. Every output is validated against standard financial formulas before being trusted, and I only prompt with test or sample data to keep any real client information out of the model.

Garage Terminal

Collector car market tracker · static dashboard refreshed by a weekly GitHub Actions scrape

`View source on GitHub: KaeJhee/garage-terminal ↗`

A Bloomberg-style dashboard that tracks market prices for 37 collector and exotic cars across JDM, exotic, European, muscle, and Chinese EV segments. A weekly GitHub Actions pipeline scrapes recent auction results with Python, commits the refreshed data, and updates the static dashboard, so it stays current with no server to run.

STACK

`Python` `BeautifulSoup` `httpx` `GitHub Actions` `JavaScript` `Chart.js` `Static hosting`

KG

ABOUT

Background and operating model

Where I've worked, how I build, and what I'm looking for.

BACKGROUND

I'm a Senior Capital Underwriter, Process Automation at Toast, where I support over $500M in capital deployments inside Toast Capital, which scaled from $100M to over $2B in annual volume during my tenure. My core work is capital decisions, customer health monitoring, and scenario planning. What sets my work apart is the systems I build alongside it: eight tools now in daily use across Toast, replacing manual workflows that used to take hours of recurring team effort each week.

I'm now on Toast's Non-R&D AI Initiatives team, bringing that build-and-validate approach to AI work in the functions outside R&D. I work through a multi-agent system I built: an orchestrator routes each task to one of eight specialized agents (workflow automation, code audit, two project management roles, devops, technical writing, product demos, and QA evaluation), each with its own system prompt, scoped tool access, and isolated context. It ships as a single plugin, so the same agents run as sub-agents in Claude Code and as Skills in Cowork, and every tool I ship goes through it to be audited, tested, and documented with its own SOP. I've also packaged reusable skills for SOP generation and product demo builds for Toast's internal plugin marketplace.

On the team side, I oversee two teammates on daily funding execution, trained five through onboarding, and built the team's new-hire program, policy and SOP index, and training video library. I'm the operational point of contact across Sales, Risk, and Operations on capital decisions, and I ran the time study and ROI analysis behind a 0-to-1 operational strategy that shipped a 70% improvement with Product, Engineering, and Operations.

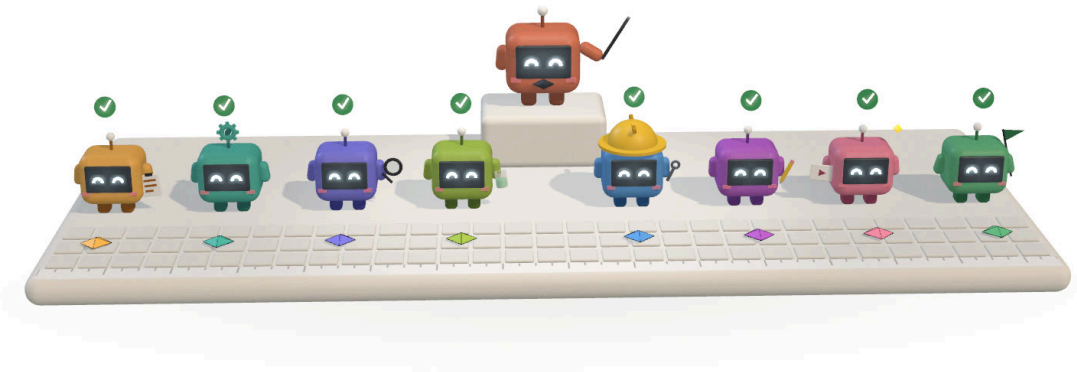
Before Toast, I spent four years at Torres Financial Services in San Antonio, moving from Junior Analyst to Portfolio Management Advisor, where I built KPI dashboards for risk, budgeting, and long-term planning and worked with compliance teams on financial regulations.

Outside of Toast, I founded Ghost Strategies LLC in November 2023, a registered investment advisory practice where I build LLM-powered financial tools: a portfolio analysis terminal for client reviews, a macro research terminal, a blind model evaluation bench, and a LangGraph earnings agent. The practice itself runs on a governed multi-agent operating model that requires human approval before anything is sent or published.

I'm based in San Antonio, Texas, and I work remote-only. I'm targeting Senior Manager-track or Senior IC roles in strategy, operations, risk, or AI-process work at companies where the AI-native operating model is the default, not the exception.

AGENT TEAM

Every tool I ship goes through this team. The orchestrator routes each task to the agent built for it, so nothing ships until it has been audited, tested, and documented.



- **Orchestrator**
  Routes each task
- **Project planning** ✓
  Plan the phases
- **Workflow automation** ✓
  Build the data pipeline
- **Code audit** ✓
  Review the code
- **QA evaluation** ✓
  Run the test cases
- **DevOps** ✓
  Set up deployment
- **Technical writing** ✓
  Write the SOP
- **Product demos** ✓
  Build the demo
- **Technical PM** ✓
  Write the changelog

HOW I WORK

01
I own the logic and architecture

→

02
The LLM writes the code

→

03
I validate every output

↻ Every change goes back through the loop. Nothing is trusted until step 03 passes.

I own the logic and architecture. I decide what a tool does, how the data flows, and what counts as a correct result.

Real example: in the Funding Verification Tool, the processor match rules (WORLDPAY needs an exact bank-file match, CHASE must differ from a known placeholder) are my business logic, isolated in code instead of buried in spreadsheet formulas.

The LLM writes the code. Claude is my primary builder, and which model fills which slot in my tools is decided task by task on recorded evidence from my own evaluation bench, blind votes and code judges, not habit.

Real example: the Portfolio Terminal's roughly 4,400 lines were written by Claude from my written handoff specifications, each with a phased build order, acceptance tests, and mock fixtures.

I validate every output before I trust it. Deterministic checks judge model output, tools ship with test harnesses, and larger builds go through adversarial multi-agent audits.

Real example: the Portfolio Terminal passed 111 automated checks, then a 130-agent adversarial audit that confirmed 56 defects, all fixed before it was trusted with real work.

No proprietary data reaches a model. I prompt with test or sample data only, so no proprietary or client information enters a model while I build.

EXPERIENCE

Toast Inc

Dec 2021 to present

Senior Capital Underwriter, Process Automation

Non-R&D AI Initiatives team · Remote, TX

Ghost Strategies LLC

Nov 2023 to present

Founder, Registered Investment Advisor

San Antonio, TX

Torres Financial Services

2017 to 2021

Portfolio Management Advisor

2019 to 2021

Junior Analyst

2017 to 2019

San Antonio, TX

EDUCATION AND CERTIFICATIONS

Bachelor of Business Administration in Finance

Degree

The University of Texas at San Antonio

San Antonio, TX

Google Data Analytics Certification

Certification

Google

CORE STACK

SQL (Snowflake) · Salesforce · Python · JavaScript · React · Google Apps Script · Hex · Excel and Google Sheets · Claude Code · Cowork · Multi-Agent Systems · LangGraph · Cloudflare Workers · Flask · FastAPI · Chart.js

KG

CONTACT

Get in touch

Best ways to reach me. I respond to email and LinkedIn within a business day.

DIRECT

EMAIL
krisgallow918@gmail.com

PHONE
832-233-9437

LINKEDIN
linkedin.com/in/kristophergallow

GITHUB
github.com/KaeJhee

LOCATION
San Antonio, TX (Remote)

SIDE PROJECT
kris@ghoststrategies.io

↓ Download resume (AI & Insights variant)

© Kristopher Gallow · Portfolio built with HTML, CSS, and JavaScript · Single-file, no framework · Source ↗